

Major Technical Systems - Guidance

This document is intended to help you understand **what we mean by a “major technical system”** in the context of a Unity game project. These are **examples and reference points**, not requirements or restrictions. You're free to pursue **any technical direction** that fits your game, as long as it meets the project requirements.

What matters most is that your proposed systems:

- Are **substantial and non-trivial**
 - **Meaningfully interact** with the rest of your game
 - Demonstrate **intentional technical design**, not just default Unity behavior
-

How Many Systems Do We Expect?

- **Teams of only Undergraduate students**
 - At least **2 major technical systems**
 - At least **1 must involve AI**, such as an NPC or agent-based decision-making system, or a system with learning or adaptation
- **Teams with a Graduate student Project Manager**
 - At least **3 major technical systems**
 - At least **1 must involve AI**, as described above

These systems do **not** need to be isolated features—they can build on each other or be tightly integrated.

What “Counts” as a Major Technical System?

A major technical system is typically:

- More than a single script or tutorial-level feature
- Configurable, extensible, or data-driven
- Responsible for coordinating or enabling meaningful gameplay behavior

For example, a simple finite state machine on its own would not be sufficient. However, a **hierarchical FSM**, or an FSM that coordinates other systems (such as navigation, animation, or utility scoring), *would* qualify.

AI & Decision-Making Systems (Examples Only)

At least one of your major systems must involve AI or decision-making. Below are **examples** of directions you might explore:

- **Agent Controllers & Behavior Architecture**
 - Behavior Trees implemented in code or via ScriptableObjects
 - FSMs that load or coordinate other decision systems
 - AI systems integrated with Unity's Animator or NavMesh
- **Utility or Scoring-Based AI**
 - Agents evaluate multiple possible actions using weighted scores
 - Decisions influenced by game state, player behavior, or internal memory
 - Often implemented in a data-driven way using ScriptableObjects
- **Adaptive or Learning Systems**
 - Dynamic difficulty adjustment based on player performance
 - Enemies that adapt tactics over time
 - Use of pre-trained models or rule-based learning integrated into Unity
- **Spatial Reasoning & Navigation**
 - Advanced use of Unity NavMesh (dynamic obstacles, area costs, runtime updates)
 - Group behaviors such as formations or flocking
Tactical positioning rather than simple "move toward player" logic
- **LLM-Driven Gameplay Systems**
 - Should go beyond a single API call
Examples include:
 - Dynamic prompt construction based on game state
 - Persistent NPC memory or dialogue history
Response caching or structured reasoning
 - **Using generative AI only to create art or assets does *not* count toward this requirement**

Again, these are **examples, not expectations**—you may pursue other AI approaches not listed here.

Non-AI Technical Systems (Examples Only)

In addition to AI, your other major systems can come from **any technical area**. Below are **examples** of Unity systems that are typically considered sufficiently complex:

- **Procedural or Dynamic Content Systems**
 - Runtime level or environment generation
 - Rule-based or noise-driven systems
 - Systems that interact with gameplay or AI

- **Multiplayer Systems**
 - Player synchronization using Unity Netcode, Mirror, or similar
 - Authority models, prediction, or replication
 - Basic lobby or session management
 - **VR / XR Systems**
 - Gameplay built around Unity's XR Interaction Toolkit
 - Hand tracking, gesture input, or spatial UI
 - Mechanics designed specifically for VR affordances
 - **Replay, Telemetry, or Metrics Systems**
 - Recording and replaying player input or game state
 - Tracking performance data for balancing or analysis
 - Exporting gameplay data
 - **Tools & Editors**
 - In-game level editors or encounter designers
 - Custom Unity Editor tools
 - Data-driven pipelines using ScriptableObjects
 - **Complex UI, Narrative, or State Systems**
 - Deep RPG-style menus (inventory, skills, dialogue)
 - Branching narratives with persistent consequences
 - Save/load systems managing multiple game states
 - **Core Architecture & Infrastructure**
 - Event-driven or service-based architecture
 - Modular systems designed for extensibility
 - Robust scene and game-state management
-

Important Clarifications (Again)

- There is no required list of systems you must implement.
- You are **not graded** on matching these examples.
- You **are graded** on:
 - Whether your systems are technically meaningful
 - Whether they support your game's design goals
 - Whether you can clearly explain what you are building and why

Your document should explain **what systems you plan** to build, **how they work** at a high level, and **why they're important** to your game.

If you can convince us that your technical plan is thoughtful, ambitious, and achievable, you are doing this correctly. Even if your approach looks very different from the examples above.